

P0891R2

Make strong_order a Customization Point!

Published Proposal, 2019-01-20

This version:

<https://github.com/atomgalaxy/a-little-order/strong-ordering.bs>

Authors:

Gašper Ažman <gasper.azman@gmail.com>

Jeff Snyder <jeff-isocpp@caffeinated.me.uk>

Audience:

LEWG, LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

The specification of ordering algorithms at the end of [\[P0768R1\]](#) does not provide the ability to provide a default order for user-defined types (since they are specified in such a way that they are not intended to be customisation points), and yet mixes in such a customisation for IEC559 floating point types. This paper suggests providing the functionality of both in a composable and principled way.

Table of Contents

1 Revision History

- 1.1 R1: Incorporated feedback from LEWG meeting in Rappersville.
- 1.2 R2: Incorporated feedback from LEWG meeting in San Diego, merged with [P0863R1]

2 Status of this paper

3 Problem Description

4 Principles

5 Discussion

5.1 Discussion of the Customisation principle

5.1.1 Is it useful?

5.1.2 Given that it is viral, is it worth the implementation effort?

5.1.3 How can I teach this?

5.1.3.1 Writing new types:

5.1.3.2 Using orderings:

5.2 Discussion of the Fallback principle

6 Questions for LEWG:

7 Proposal

7.1 Remove the `strong_equal` and `weak_equal` comparison algorithms

7.2 Make `strong_order` and friends customisation points

7.3 Replace the IEC559 rule in `strong_order` with separate overloads

7.4 Make the weaker customisation points call stronger customizatoin points

7.5 Add fallback functions

8 Proposed Wording

9 Acknowledgments

Appendix A: Proof `strong_order` is not a valid customisation point

Index

Terms defined by this specification

References

Normative References

Informative References

§ 1. Revision History

§ 1.1. R1: Incorporated feedback from LEWG meeting in Rappersville.

The feedback was:

- Remove the floating point exception (bullet 1.1) as R0 recommended, since Unicode strings, etc., are a possible rationale.
- Do not propose making existing `*_order` functions "customisation points" (as used in [\[P0551R3\]](#)).
- Add a new `default_order` customisation point, along with a bikeshedding section on its actual name, with the behaviour:
 - It has the IEC 559 behavior from bullet 1.1 of `strong_order`
 - It is defined for all (other) floating-point types; it is implementation-defined whether it is consistent with the partial order from the comparison operators. (Implementations should do this.)
 - It is a customisation point (à la [\[P0551R3\]](#)).
- Investigate the possibility of adding Lawrence's weak order (from [\[P0100R2\]](#)) for floating-point numbers (which did not make it in with spaceship).

§ 1.2. R2: Incorporated feedback from LEWG meeting in San Diego, merged with [\[P0863R1\]](#)

Feedback in San Diego was:

- We want to solve `std::set<T>` for not obviously comparable `T` (0/6/4/1/1).
- We want to solve `std::unordered_set<T>` for not obviously hashable `T`. (Out of scope of this paper)
- We need to address the fact that containers need to provide specializations of `strong_order` if it is a customisation point.
- We want `strong_order` to be the customisation point (Herb's proposed poll) - never taken.

§ 2. Status of this paper

R2 of this paper is a merge of its R1 and Jeff Snyder's [\[P0863R1\]](#), and is synchronized with Barry Revzin's [\[D1186R1\]](#). It incorporates all feedback for R0 and R1, and presents a coherent design for the library components of `<=>`.

§ 3. Problem Description

This paper is a proposal to amend the *comparison algorithms* that have been voted into the working draft as part of [\[P0768R1\]](#).

As worded, the `strong_order` comparison algorithm provides:

- a strong ordering for some types that do not have it provided by `<=>` (IEC559 floating point types),
- a way of generating a value of type `strong_ordering` by falling back to the `==` and `<` operators if `<=>` is not callable or does not result in a `strong_ordering`.

However, the authors of this paper and the author of [\[P0515R3\]](#) intended for `strong_order` to be a customisation point, and it is not currently usable as such.

In the standard, we need to provide the ability to customize `strong_order` and a way to fall back to `==` and `<`. This paper argues that these are fundamentally incompatible, and cannot be served by the same function.

[\[P0891R0\]](#) and [\[P0891R1\]](#) of this paper presented the case for having a customisation point for ordering in the language. This paper only summarizes the currently relevant bits of that discussion. For wider context, the reader should refer to the previous revisions, as well as to [\[P0863R1\]](#).

§ 4. Principles

To arrive at the final design of this paper, we used the following principles:

- **Consistency:** `*_order` algorithms should behave consistently with `<=>` by default.
- **Weakening:** If a type has a given order, then it also has all weaker orders.
- **Ambiguity of Legacy:** The ordering category provided by the legacy comparison operators is ambiguous. The standard currently expects merely that `<` provides a weak ordering.
- **Customisation:** As `<=>` is a customisation point for the natural ordering on a type, so `strong_order` is a customisation point for an arbitrary strong order on that type.
- **Fallback:** We need convenient functions to get `*_ordering` types from legacy comparison operators.

From these principles, these corollaries follow:

- **Corollary 1:** from [Consistency](#) it follows that, if `<=>` provides a given order `X`, then the default implementation of `X_order` should use it.

Example: `strong_order(1, 2)` should be equivalent to `1 <=> 2`, since `int`'s `operator<=>` provides a strong ordering. `weak_order(1, 2)` likewise, since `1 <=> 2` is convertible to `strong_ordering`.

- **Corollary 2:** from [Consistency](#), [Weakening](#) and [Customisation](#), it follows that:
 - if `<=>` does not provide a strong enough order, but a stronger ordering function (e.g. `strong_order` for `weak_order`, `weak_order` for `partial_order`) is available, then the weaker ordering function must fall back on it (this works recursively, so if `strong_order` is defined, so is `partial_order`).

Example: given `struct C { partial_ordering operator<=>(C const&) const; }; strong_ordering strong_order(C const&, C const&); weak_order(c1, c2)` should return `strong_order(c1, c2)`, because `c1 <=> c2` does not provide a sufficiently strong order.
- **Corollary 3:** from [Consistency](#) and [Fallback](#), it follows that if `<=>` exists, the fallback function should use it in preference of legacy comparison operators.
- **Corollary 4:** From [Fallback](#) and [Ambiguity of Legacy](#), it follows that by using a fallback function, the user is asserting that `<` and `==` implement the corresponding ordering.
- **Corollary 5:** From [Customisation](#) and the definition of *customisation point*, it follows that `*_order` functions must be specified as *does not participate in overload resolution* when they cannot be synthesized according to the above rules.
- **Corollary 6:** From [Corollary 4](#) and [Corollary 5](#) it follows that the fallbacks and the customisation points cannot be the same functions, as their roles are fundamentally incompatible. To illustrate: the fallback function for synthesizing `strong_ordering` must exist whenever `<` and `==` do, but the `strong_order` customisation point must not be synthesized purely from those.

In addition, we have made the following observations:

- Given [\[D1186R1\]](#) provides a method of generating a `<=>` operator from `<` and `==`, we should (due to [Consistency](#)) defer to that method for synthesizing the fallback functions.
- If `strong_order` is a customisation point for providing a stronger order than `<=>` does, data structures that recurse to their contents for `<=>` should also recurse for `strong_order`.
- Given that the `<=>` operator is not being provided for any of the types in the standard library C++20, we are deferring proposing this until such time as `<=>` is proposed for them as well.

§ 5. Discussion

§ 5.1. Discussion of the Customisation principle

The arguments around whether `strong_order` should be a customisation point have centered around the following questions:

- Is it even useful?
- Given that it is viral, is it worth the implementation effort?
- Should all of the comparison algorithms be customisation points?
- How can this be taught?

§ 5.1.1. Is it useful?

There are many algorithms and data structures that do not care about ordering per se, but do require some arbitrary order. Examples of these include `set`, `map`, and fast implementations of set algorithms (union, intersection, difference, etc), which are typically based on ordered sequences.

There also exist types for which there is no natural strong ordering, such as `float` and `complex`. Even though these types do not have a natural strong ordering, it is nevertheless very useful to provide an arbitrary strong ordering for them, e.g. so that they can be used in containers such as `set` and `map`. Having an arbitrary strong ordering is in fact so useful that *Stepanov* and *Mc Jones* included it in their definition of *Regular* in *Elements of Programming*.

In order to make it possible to use IEC559 floating-point types in `sets` and `maps`, the existing `strong_order` comparison algorithm includes a special rule for IEC559 floating point types. This works well for simple sets of floating point numbers, but it breaks down quickly when we try more complicated examples such as `set<optional<float>>` or `set<pair<float, int>>`. To make these examples work, we need to *customise* `strong_order` for `optional`, `pair` and other container types, and for that to be permitted `strong_order` must be a customisation point.

Having an agreed-upon customisation point for an arbitrary order is long overdue. It is time to define what that customisation point is, so it can be provided and used consistently across C++ libraries and applications. That name should be `strong_order`, since `default_order` as a distinct customisation point was rejected by LEWG in San Diego in favor of the design in R0 of this paper.

§ 5.1.2. Given that it is viral, is it worth the implementation effort?

Given that any customisation point we designate has to potentially be overloaded for all the generic wrappers in the standard library, is it worth the effort of defining them?

This paper does not propose any such wrappers, because the standard library does not even support `operator <=>` for such wrapper types yet. This paper only proposes that `strong_order` be established as a customisation point.

§ 5.1.3. How can I teach this?

For types that endeavour to be *Regular* or something like it (as in, have sane value semantics), do the following.

§ 5.1.3.1. Writing new types:

- `operator==` defines what "strong equality" means. This is because copies must compare equal, and `==` should be the finest relation that observes this rule. EOP defines it as "must represent the same value in the domain of the type."
- if `(x <=> y) == 0` if and only if `x == y`, then `operator<=>` should return `strong_ordering`, otherwise it should return `weak_ordering` (unless it only defines a partial order, and then it must return `partial_ordering` anyway).
- If your type does not provide an order, you should still provide an overload of `strong_order` that defines an arbitrary order consistent with `==`.
- If you are obeying the rule of zero, writing `auto operator<=>(...) = default` will give you the correct `==` and `<=>` without any additional work. In most cases, this should already be enough, since `<=>` will result in a `strong_ordering`.
- If you are writing a generic wrapper, or your constituent types have an overloaded `strong_order`, you must then also define `strong_order` to be:

```
strong_ordering strong_order(T x, T y) {  
    if (auto ord = strong_order(x.a, y.a); is_neq(ord)) { return ord; }  
    if (auto ord = strong_order(x.b, y.b); is_neq(ord)) { return ord; }  
    // ... for all members  
    return strong_ordering::equal;  
}
```

§ 5.1.3.2. Using orderings:

Generic algorithms should require and call `strong_order(x, y)`, `weak_order(x, y)` or `partial_order(x, y)` (depending on the ordering requirements of the algorithm) after a `using namespace std;`. These will find the most appropriate ordering available, or SFINAE away if no suitable ordering is available. For example, if an algorithm calls `weak_order` on a type which has `<=>` returning `partial_order`, does not provide an overload of `weak_order`, but does overload `strong_order`, then `strong_order` will end up being called. The intent is that these "always do the right thing".

§ 5.2. Discussion of the Fallback principle

The arguments on whether we need to provide the fallback functions for legacy types center around a few questions:

- How do we make it easy to define `<=>` for types which have members which only define `<` and `==`, but not `<=>`?
- How should generic code work with types that may or may not define `<=>`, *but are assumed to have the requisite preconditions on `<` and `==`*?
- In a context where we need to pass an ordering type as a parameter to a function, but we need to obtain it from a legacy type, how do we do that?

[D1186R1] makes the first of these trivial, and defines the algorithms necessary to make use of the legacy type's `==` and `<` operators. Helpfully, it defines the `3WAY<>(X, Y)` specification macro, which contains the core of its fallback logic.

The second two questions boil down to the same thing: if we have a type that implements `==` and `<`, and we know it has a particular ordering, how do we convert that into a value of the appropriate `X_ordering` type? We expect this to become a common problem, and therefore the standard should provide convenience functions that call `==` and `<` and return an `X_ordering` value representing the result. In specifying these functions, we can re-use the `3WAY<>(X, Y)` from [D1186R1], both for convenience and consistency.

However, as C++ applications and libraries are updated to implement `operator <=>` for their types, the need for these fallback functions should diminish. On the other hand, the customisation points are a feature with permanent utility. Due to customziation points having a longer term utility, they should get the `X_order` names, and we should find new names for fallback functions.

§ 6. Questions for LEWG:

1. Is there consensus that the [Customisation](#) premise (as above) is correct and should be adopted?
 - If not, then should the IEC559 exception bullet in the current working draft be removed?
2. Is there consensus that the [Fallback](#) premise (as above) is correct and should be adopted?
 - Rejecting this means that we are rejecting the rationale for their inclusion in the current working draft and that they should be removed.

(The [Consistency](#), [Weakening](#) and [Ambiguity of Legacy](#) are true from a good design, mathematical, and empirical point of view, respectively, so they are not subject to poll).

§ 7. Proposal

§ 7.1. Remove the `strong_equal` and `weak_equal` comparison algorithms

If we accept [\[P1185R0\]](#)'s rationale for not calling `<=>` when the user only needs equality (as EWG did in San Diego), we should also avoid doing so in the library. However, without making assumptions about the behaviour of `==`, there is no way to implement `strong_equal` and `weak_equal` without calling `<=>`.

Therefore, we propose removing the `strong_equal` and `weak_equal` algorithms.

§ 7.2. Make `strong_order` and friends customisation points

For the functions `strong_order`, `weak_order` and `partial_order` we propose to:

- Replace the functions with [Customisation Point Objects](#).
- Remove the fallbacks to `==` and `<`
- Have the functions not participate in overload resolution (instead of being defined as deleted) if there is no strong order available.

§ 7.3. Replace the IEC559 rule in `strong_order` with separate overloads

Since `strong_order` is a customisation point, the strong order for floating point types should be provided as overloads of `strong_order` rather than being baked into the generic `strong_order`

function.

We propose that the bullet point regarding `IEC559` be removed from the `strong_order` algorithm, and that the following three overloads for `strong_order` are added:

- `strong_order(float, float);`
- `strong_order(double, double);`
- `strong_order(long double, long double);`

These overloads should implement the `totalOrder` operation as specified in ISO/IEC/IEEE 60559, and should not participate in overload resolution if `numeric_limits<T>::is_iec559` is `false` for the type of their arguments.

§ 7.4. Make the weaker customisation points call stronger customisation points

To handle cases such as where a type provides a `partial_ordering` from `<=>` and a `strong_order` overload, if a customisation point cannot get an appropriate result from calling `<=>`, it should try to call a customisation point for an order stronger than its own. Specifically:

- `weak_order` should fall back to calling `strong_order`
- `partial_order` should fall back to calling `weak_order`

§ 7.5. Add fallback functions

To avoid losing the functionality of the existing comparison algorithms, we propose to keep them under new names, with the following changes to their behaviour:

- They will, in preference to anything else, return the result of calling the corresponding customisation point.
- They will not attempt to call `<=>` directly, as this is called by the customisation point.
- They will fall back to `==` and `<` using the language mechanism proposed by [\[D1186R1\]](#) instead of calling `==` and `<` directly.

As per the original comparison algorithms, they will be defined as deleted if neither the corresponding customisation point nor the `==` and `<` operators are available.

Following the same reasoning as in [§7.1 Remove the strong_equal and weak_equal comparison algorithms](#), there will be no fallback functions for `strong_equality` and `weak_equality`.

We propose naming the fallback functions `assumed_strong_order`, `assumed_weak_order` and `assumed_partial_order`.

Other options to consider:

- `assumed_X_order`
- `X_order_assume`
- `assume_X_order`
- `fallback_X_order`
- `X_order_fallback`

See the [fallback](#) discussion on why the `X_order` names should be given to the customisation points.

§ 8. Proposed Wording

Change the current `strong_order` to `assumed_strong_order` (but see bikeshedding section):

```
template<class T> constexpr strong_ordering assumed_strong_order(const T&  
a, const T& b);
```

- # Effects: Compares two values and produces a result of type `strong_ordering`:
 - ~~(1.1) If `numeric_limits<T>::is_iec559` is true, returns a result of type `strong_ordering` that is consistent with the `totalOrder` operation as specified in ISO/IEC/IEEE 60559.~~
 - (#.1) If the expression `strong_order(a, b)` is well-formed and convertible to `strong_ordering`, return the result of the expression,
 - ~~(1.2) Otherwise, returns `a <=> b` if that expression is well-formed and convertible to `strong_ordering`.~~
 - (#.2) Otherwise, if the expression `3WAY<strong_ordering>(a, b)` is well-formed, return the result of the expression,
 - ~~(1.3) Otherwise, if the expression `a <=> b` is well-formed, then the function is defined as deleted.~~
 - (1.4) Otherwise, if the expressions `a == b` and `a < b` are each well-formed and convertible to `bool`, then
 - (1.4.1) if `a == b` is true, returns `strong_ordering::equal`;
 - (1.4.2) otherwise, if `a < b` is true, returns `strong_ordering::less`;

- ~~(1.4.3) otherwise, returns `strong_ordering::greater`.~~

- (#.53) Otherwise, the function shall be defined as deleted.

Change the current `weak_order` to `assumed_weak_order` (but see bikeshedding section):

```
template<class T> constexpr weak_ordering assumed_weak_order(const T& a,
const T& b);
```

- # Effects: Compares two values and produces a result of type `weak_ordering`:
 - (#.1) If the expression `weak_order(a, b)` is well-formed and convertible to `weak_ordering`, return the result of the expression,
 - ~~(2.1) Returns `a <=> b` if that expression is well-formed and convertible to `weak_ordering`.~~
 - (#.2) Otherwise, if the expression `3WAY<weak_ordering>(a, b)` is well-formed, return the result of the expression,
 - ~~(2.2) Otherwise, if the expression `a <=> b` is well-formed, then the function is defined as deleted.~~
 - ~~(2.3) Otherwise, if the expressions `a == b` and `a < b` are each well-formed and convertible to `bool`, then

 - ~~(2.3.1) if `a == b` is true, returns `weak_ordering::equivalen`;~~
 - ~~(2.3.2) otherwise, if `a < b` is true, returns `weak_ordering::less`;~~
 - ~~(2.3.3) otherwise, returns `weak_ordering::greater`.~~~~
 - (#.43) Otherwise, the function shall be defined as deleted.

Change the current `partial_order` to `assumed_partial_order` (but see bikeshedding section):

```
template<class T> constexpr partial_ordering assumed_partial_order(const T&
a, const T& b);
```

- # Effects: Compares two values and produces a result of type `partial_ordering`:
 - (#.1) If the expression `partial_order(a, b)` is well-formed and convertible to `partial_ordering`, return the result of the expression,
 - ~~(2.1) Returns `a <=> b` if that expression is well-formed and convertible to `partial_ordering`.~~
 - (#.2) Otherwise, if the expression `3WAY<partial_ordering>(a, b)` is well-formed, return the result of the expression,

- (2.2) Otherwise, if the expression $a \leq b$ is well-formed, then the function is defined as deleted.
- (2.3) Otherwise, if the expressions $a == b$ and $a < b$ are each well-formed and convertible to bool, then
 - (2.3.1) if $a == b$ is true, returns `partial_ordering::equivalent`;
 - (2.3.2) otherwise, if $a < b$ is true, returns `partial_ordering::less`;
 - (2.3.3) otherwise, returns `partial_ordering::greater`.
- (#.43) Otherwise, the function shall be defined as deleted.

From section 24.x.4, Comparison Algorithms [cmp.alg], remove `strong_equal` and `weak_equal`:

```
template<class T> constexpr strong_equality strong_equal(const T& a, const T& b);
```

- ~~4 Effects: Compares two values and produces a result of type `strong_equality`:~~
 - (4.1) Returns $a \leq b$ if that expression is well-formed and convertible to `strong_equality`.
 - (4.2) Otherwise, if the expression $a \leq b$ is well-formed, then the function is defined as deleted.
 - (4.3) Otherwise, if the expression $a == b$ is well-formed and convertible to bool, then
 - (4.3.1) if $a == b$ is true, returns `strong_equality::equal`;
 - (4.3.2) otherwise, returns `strong_equality::nonequal`.
 - (4.4) Otherwise, the function is defined as deleted.

```
template<class T> constexpr weak_equality weak_equal(const T& a, const T& b);
```

- ~~5 Effects: Compares two values and produces a result of type `weak_equality`:~~
 - (5.1) Returns $a \leq b$ if that expression is well-formed and convertible to `weak_equality`.
 - (5.2) Otherwise, if the expression $a \leq b$ is well-formed, then the function is defined as deleted.
 - (5.3) Otherwise, if the expression $a == b$ is well-formed and convertible to bool, then
 - (5.3.1) if $a == b$ is true, returns `weak_equality::equivalent`;
 - (5.3.2) otherwise, returns `weak_equality::nonequivalent`.
 - (5.4) Otherwise, the function is defined as deleted.

Then add the `strong_order` customisation point object:

(#) The name `strong_order` denotes a customisation point object. The expression `std::strong_order(E, F)` for some subexpressions `E` and `F` with type `T` is expression-equivalent to:

(#.1) `strong_order(E, F)` if it is a valid expression,

(#.2) Otherwise, `E <=> F` if it is a valid expression and its type is convertible to `strong_ordering`,

(#.3) Otherwise, `std::strong_order(E, F)` is ill-formed. [Note: This case can result in substitution failure when `strong_order(E, F)` appears in the immediate context of a template instantiation. --end note]

[Note: Whenever `std::strong_order` is a valid expression, its type is convertible to `strong_ordering`. --end note]

Add overloads for IEC559 types:

(#) `strong_order(T, T)` when `T` is a type for which `std::numeric_limits<T>::is_iec559` is `true` shall be a valid expression, whose value shall be consistent with the `totalOrder` operation as specified in ISO/IEC/IEEE 60559.

Add the `weak_order` customisation point object:

(#) The name `weak_order` denotes a customisation point object. The expression `std::weak_order(E, F)` for some subexpressions `E` and `F` with type `T` is expression-equivalent to:

(#.1) `weak_order(E, F)` if it is a valid expression,

(#.2) Otherwise, `E <=> F` if it is a valid expression and its type is convertible to `weak_ordering`,

(#.3) Otherwise, `std::strong_order(E, F)` if it is a valid expression,

(#.4) Otherwise, `std::weak_order(E, F)` is ill-formed. [Note: This case can result in substitution failure when `weak_order(E, F)` appears in the immediate context of a template instantiation. --end note]

[Note: Whenever `std::weak_order` is a valid expression, its type is convertible to `weak_ordering`. --end note]

Add the `partial_order` customisation point object:

(#) The name `partial_order` denotes a customisation point object. The expression `std::partial_order(E, F)` for some subexpressions `E` and `F` with type `T` is expression-equivalent to:

(#.1) `partial_order(E, F)` if it is a valid expression,
(#.2) Otherwise, `E <=> F` if it is a valid expression and its type is convertible to `partial_ordering`,
(#.3) Otherwise, `std::weak_order(E, F)` if it is a valid expression,
(#.4) Otherwise, `std::partial_order(E, F)` is ill-formed. [Note: This case can result in substitution failure when `partial_order(E, F)` appears in the immediate context of a template instantiation. --end note]

[Note: Whenever `std::partial_order` is a valid expression, its type is convertible to `partial_ordering`. --end note]

§ 9. Acknowledgments

We would like to thank

- **Roger Orr** for bringing this to our attention;
- **Thomas Köppe** for his valuable comments, review, and most of all some extremely clear and laconic wording;
- **Sam Finch** for *thoroughly* breaking the original examples, some example code, great substantive comments, and pointing out that the current definition actually breaks types that define a partially-ordered set of comparison operators;
- **Richard Smith** for further fixing my example in light of Concepts, and example code.
- **Herb Sutter and Walter Brown** for providing (mutually opposing) guidance on customisation points.
- **Louis Dionne** for great comments on the structure of the paper and how to bring the focus where it needs to be;
- **Walter Brown** for representing the paper at a committee meeting when Gašper could not make it in person, and guidance with direction;
- **Barry Revzin** for representing the paper at a committee meeting when Gašper could not make it in person, and his various papers that touch ours, and for pre-reviewing our wording and suggesting we make the customisation points CPOs, and reworking his paper to support the `3WAY<strength>(x, y)` syntax.
- **Herb Sutter** for his comments and support for getting ordering right.

And, *again*, a special thank-you to Walter Brown, who, with his final lightning talk in Bellevue, reminded us to remember whose shoulders we are standing on.

Thank you all!

§ Appendix A: Proof `strong_order` is not a valid customisation point

Say we have a template struct representing the Gaussian integers, with a *natural order* defined by the Manhattan distance from `0+0i`. This struct still defines a `std::strong_order` to model **Regular**.

Note: The **Regular** above refers to the *Elements of Programming* concept, not the ISO C++ **Regular**, which is weaker.

Note: There is no natural order on Gaussian integers, but humor this example, please.

```

namespace user {
    template <typename T>
    struct gaussian {
        static_assert(std::is_integral_v<T>);
        T re;
        T im;

        constexpr std::strong_equality operator==(gaussian const& other) const {
            return re == other.re && im == other.im;
        }
        constexpr std::weak_ordering operator<=>(gaussian const& other) const {
            return (*this == other) ? std::weak_ordering::equal
                : (abs(*this) == abs(other)) ? std::weak_ordering::less
                : abs(*this) <= abs(other) ? std::weak_ordering::less : std::weak_ordering::greater;
        }

        friend constexpr T abs(gaussian const&) {
            using std::abs;
            return abs(re) + abs(im);
        }

        friend constexpr std::strong_ordering strong_order(gaussian const& x,
                                                            gaussian const& y) {
            // compare lexicographically
            return std::tie(x.re, x.im) <=> std::tie(y.re, y.im);
        }
    };
}

```

Consider a transparent ordering operator for `map`:

```

struct strong_less
    template <typename T, typename U>
    bool operator()(T const& x, U const& y) {
        using std::strong_order; // use ADL
        return strong_order(x, y) < 0;
    }
    using is_transparent = std::true_type;
};

```

Also say we had a type with an implicit conversion to our `gaussian`:

```
template <typename T>
struct lazy {
    std::function<T()> make;
    operator T() const { return make(); }
};
```

This function now fails to compile, because the chosen `std::strong_order` is deleted.

```
bool exists(lazy<gaussian<int>> const& x,
            std::set<gaussian<int>, strong_less> const& in) {
    /* imagine this being a template in both parameters - it's pretty normal
    return in.count(x);
    }
```

The std-provided `std::strong_order` is deleted because it cannot be synthesized from `gaussian`'s `operator<=>`. The reason it is chosen over the friend function, however, is because the standard template matches better than the friend which would require an implicit conversion.

If the std-provided `std::strong_order` did not participate in overload resolution, however, this example would work just fine.

§ Index

§ Terms defined by this specification

[Ambiguity of Legacy](#), in §4

[Consistency](#), in §4

[Corollary 1](#), in §4

[Corollary 2](#), in §4

[Corollary 3](#), in §4

[Corollary 4](#), in §4

[Corollary 5](#), in §4

[Corollary 6](#), in §4

[Customisation](#), in §4

[Fallback](#), in §4

[Weakening](#), in §4

§ References

§ Normative References

[D1186R1]

(Untitled). URL: <https://wg21.link/d1186r1>

[P0551R3]

Walter E. Brown. [Thou Shalt Not Specialize std Function Templates!](#). 16 March 2018. URL: <https://wg21.link/p0551r3>

§ Informative References

[P0100R2]

Lawrence Crowl. [Comparison in C++](#). 27 November 2016. URL: <https://wg21.link/p0100r2>

[P0515R3]

Herb Sutter, Jens Maurer, Walter E. Brown. [Consistent comparison](#). 10 November 2017. URL: <https://wg21.link/p0515r3>

[P0768R1]

Walter E. Brown. [Library Support for the Spaceship \(Comparison\) Operator](#). 10 November 2017. URL: <https://wg21.link/p0768r1>

[P0863R1]

Jeff Snyder. [Fixing the partial_order comparison algorithm](#). 8 October 2018. URL: <https://wg21.link/p0863r1>

[P0891R0]

Gašper Ažman. [Let strong_order Truly Be a Customization Point!](#). 10 February 2018. URL: <https://wg21.link/p0891r0>

[P0891R1]

Gašper Ažman. [Everyone Deserves a Little Order](#). 27 October 2018. URL: <https://wg21.link/p0891r1>

[P1185R0]

Barry Revzin. [<=> != ==](#). 7 October 2018. URL: <https://wg21.link/p1185r0>